Windowed Fog Aggregation and Batched Serverless Ingestion for a Two-Pond Aquaculture Water-Quality Monitor

Anjaneya Reddy Gurram
Student ID: 24288853
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x24288853@student.ncirl.ie

***Abstract*—Pond fish farms still rely on manual test-kit readings taken once or twice a day, leaving water-quality excursions such as an overnight oxygen crash undetected until fish are already stressed. This paper presents a continuous two-pond monitoring pipeline. Five water-quality sensors per pond feed a fog gateway that windows and aggregates readings and evaluates threshold alerts locally, forwarding only compact summaries to the cloud. A scalable serverless backend on Amazon SQS, AWS Lambda, and DynamoDB ingests those summaries and serves a live dashboard through API Gateway. Each layer is justified against alternatives, and the system is deployed to a real AWS account rather than a local emulator. An automated suite of 156 tests passes, and end-to-end verification against the live deployment confirmed every pipeline health check reporting healthy, fresh readings arriving within seconds, and per-pond metrics and alerts rendering correctly on the deployed dashboard. A pre-deployment audit additionally caught three defects a local emulator had masked.**

***Keywords*—aquaculture monitoring, fog computing, Internet of Things, serverless computing, Amazon SQS, Amazon DynamoDB, AWS Lambda, water quality sensing**

I. Introduction

Most small and mid-sized fish farms still assess pond condition the way Boyd's standard reference on aquaculture water quality describes: a technician draws a sample, walks it to a test kit or meter, and records a handful of readings once or twice a day [1]. That cadence is a reasonable match for water chemistry that changes slowly, but dissolved oxygen and ammonia in a stocked pond do not always change slowly: a warm, still night can drive oxygen down and ammonia up over a few hours, well inside the gap between two manual readings, and a farm that only samples at sunrise has no way to see the excursion until fish are already stressed. Aquaculture's own scale makes that gap increasingly expensive to leave unaddressed: the United Nations' most recent global fisheries assessment records aquaculture overtaking capture fisheries as the majority source of aquatic animal production for the first time, a shift it frames as raising rather than lowering the stakes on how reliably that production is managed [2].

Wireless sensing has been proposed as a fix for exactly this gap for close to two decades. An early deployment on the Irish coast, SmartCoast, validated a waterproof multi-sensor node reporting temperature, pH, and dissolved oxygen over ZigBee specifically to close the interval between manual water-quality checks [3]. Simbeye and Yang later built a purpose-built aquaculture pond network on the same wireless-sensor premise, adding an SMS alert path so a farm owner away from the site would still learn of a threshold breach promptly rather than at the next scheduled visit [4]. More recent aquaculture IoT work has pushed a further step past raw wireless sensing toward distributing the processing itself: Encinas et al. describe a distributed water-quality monitoring architecture for aquaculture that pushes summarisation toward the sensing tier rather than shipping every raw reading to a single collection point [5]. That distributed-processing instinct is the same one fog computing formalises more generally: a compute tier positioned physically close to the sensors, intercepting and reducing a data stream before it reaches a distant service, precisely because turning ten seconds of raw samples into one short summary near the source avoids paying network and storage cost for readings that individually rarely matter [6]. Behind that fog tier, this project still relies on cloud infrastructure in the sense NIST formalised for on-demand, elastically provisioned computing [7], rather than a second fixed installation sized for a worst case that mostly does not occur.

This report documents a system built on that two-tier premise for a concrete two-pond fish farm: two Docker-hosted ponds, five sensor types apiece, one fog gateway per deployment reducing every reading before it ever leaves the local network segment, and a cloud-side pipeline that only does work when a window actually closes with data in it. What this report sets out to verify, not merely describe, comes down to four separate claims. The fog gateway's windowing and threshold logic has to be shown actually cutting the volume of traffic reaching the cloud, not merely capable of doing so in principle. The serverless ingestion path has to be shown taking in a burst of several windows closing at once as queue backlog rather than as failed or throttled invocations. The single-thread, per-sensor sampling design has to be shown

surviving concurrent load without dropping a reading. And the whole pipeline has to be shown running against billed AWS infrastructure, not only against the LocalStack stand-in most of the implementation work was checked against day to day. Each of those four claims is picked up again with concrete evidence in Section III below. The architecture behind them, together with the choices weighed at each layer and the system's security posture, is the subject of Section II immediately following; Section III covers the software implementation, its automated tests, continuous integration, the real deployment together with the defects a pre-deployment audit corrected, and the live-endpoint verification of the deployed system against both the test suite and evidence gathered from the running endpoint; and Section IV closes with a critical assessment of the design's trade-offs and where it could go next.

II. System Architecture and Design

Fig. 1 lays out the deployed system's moving parts: ten sensor containers and the fog gateway sharing one EC2 instance, an SQS queue, two independently deployed Lambda functions, a DynamoDB table, and a browser-facing S3 bucket reached through API Gateway.

A. Sensor and Fog Layer

Every sensor container in this design runs one process per sensor-and-pond pair (ten processes total for two ponds of five sensor types), and each process holds two independent timing deadlines inside a single thread rather than splitting sampling and dispatch across separate threads or operating-system processes. A tight loop compares the current time against a next-sample deadline and a next-dispatch deadline on every iteration, advances a bounded random walk and appends a reading whenever the first has passed, flushes the accumulated batch to the fog gateway whenever the second has passed, and sleeps for whatever time remains until whichever deadline is nearer. The two intervals are independently configurable per container: dissolved oxygen, for instance, can sample every couple of seconds while only dispatching every ten, so several readings genuinely accumulate into one network round trip rather than one POST per sample. This single-thread, dual-deadline design was chosen over a two-thread producer/consumer split specifically because a sensor process here has no shared mutable state to protect between its sampling and dispatch concerns; introducing a second thread would only add synchronisation a design with one thread and two deadlines does not need. Each sensor's bounded walk is parameterised against a realistic physical range: water temperature drifts within ten to thirty-four degrees Celsius around a twenty-four-degree start, dissolved oxygen within one to twelve milligrams per litre around seven, pH within 5.5 to 9.0 around 7.2, ammonia within zero to two parts per million around 0.15, and feed dispensed within zero to five hundred grams around a hundred and twenty. Each dispatch call's request body is a small JSON object naming the sensor type, the pond's site identifier, the reading's unit, and an array of the timestamped values accumulated since the previous dispatch; even with several samples batched into one call at the default sampling and dispatch intervals, the whole body comes to a few hundred bytes, comfortably under a kilobyte, so batching sampling and dispatch onto separate deadlines never risks an oversized request.

The fog gateway itself is a plain Java HTTP server built on the JDK's standard library, not a framework. Incoming readings land in a concurrent hash map keyed on sensor type and pond, and every ingest call reduces to a single call to that map's own atomic per-key merge operation; there is no explicit lock, no atomic-reference fencing, and no dedicated draining thread anywhere in the buffering path. Correctness instead rests on two properties working together: the JDK's own per-key merge guarantee, and an accumulator value type that is genuinely immutable, so every combine call returns a fresh instance rather than mutating either argument in place, closely matching the discipline Lea's account of the Java concurrency utilities describes as necessary for that package's non-blocking guarantees to hold in practice [8]. A ten-second scheduled task retires the whole buffer by swapping in a brand-new empty map and iterating the retired one undisturbed, so a flush in progress never contends with concurrent ingest calls for the same reference. Each retired sensor-and-pond bucket becomes one window aggregate carrying its reading count, minimum, maximum, rounded average, and the last-arrived value reported separately as *latest*. Five threshold rules, each built through a small fluent chain (specifying a sensor type, a field to test, a comparison direction, and a limit, in that order, so an incomplete rule cannot be constructed), are then evaluated against every closed window: a dissolved-oxygen average below 4.0 milligrams per litre flags hypoxia risk, an ammonia average above 0.5 parts per million flags toxicity risk (a bound well inside the range Ip and Chew's review of ammonia toxicity in fish identifies as physiologically damaging under sustained exposure [9]), a water-temperature average above 30.0 degrees flags heat stress, and pH average readings above 8.5 or below 6.5 flag alkaline and acidic risk respectively, the two pH rules coexisting against the same sensor type without either one shadowing the other. Every request the gateway serves, ingest included, is routed through a single dispatcher registered once and matched at request time against an ordered list of path predicates, rather than one registration per route, discussed further alongside the dashboard's identical mechanism in Section II-C.

B. Backend: Queue, Function, and Store

The fog gateway never calls the ingestion Lambda directly. Every window's finished summaries are placed on an SQS queue instead, a decoupling choice with a long pedigree in distributed messaging design, following Hohpe and Woolf's catalogue of enterprise integration patterns, which frames a queue exactly this way, as the mechanism that lets a sender's request rate and a receiver's processing rate vary independently of one another without either side blocking on the other [10]. That decoupling means a burst of pond activity closing several sensor windows in the same cycle becomes queue backlog

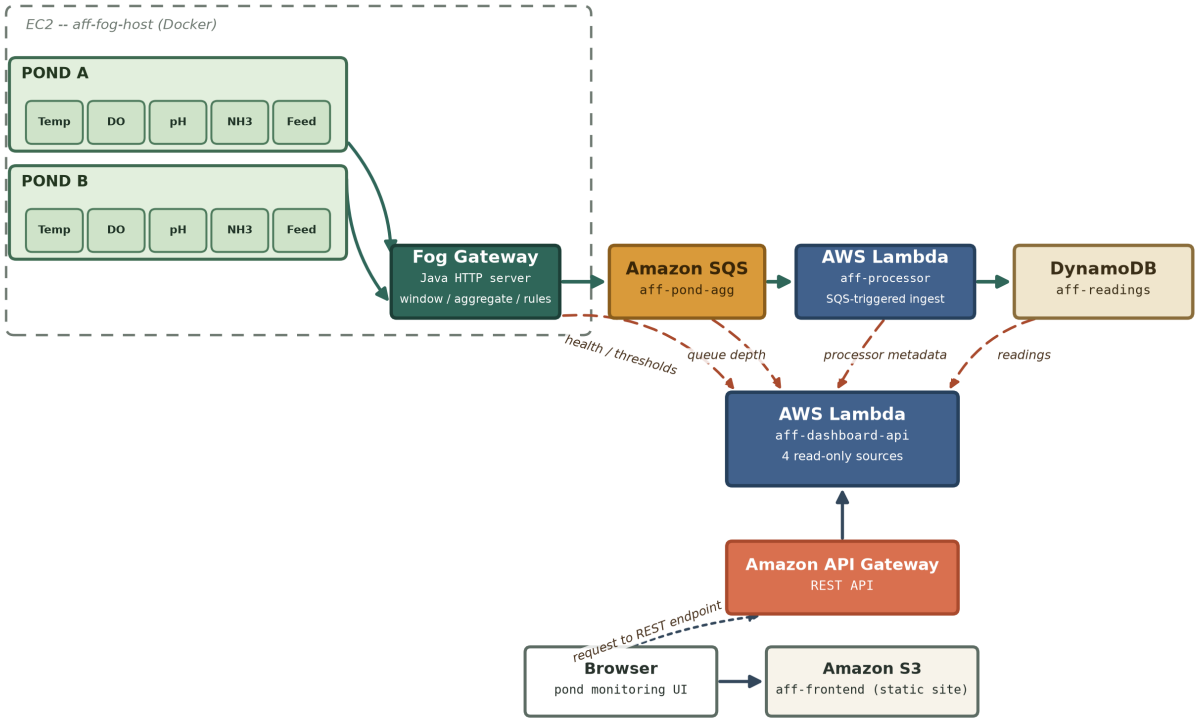

Fig. 1. Deployed topology. Ten sensor containers (two ponds, five sensor types each) share one EC2 instance with the co-located fog gateway, which batches window aggregates onto SQS. Lambda aff-processor consumes the queue into DynamoDB; Lambda aff-dashboard-api, invoked through API Gateway, pulls four of its own read-only sources at request time (dashed edges): the fog gateway's health and thresholds endpoints, the queue's current depth, aff-processor's own deployment metadata, and DynamoDB itself.

rather than a wave of failed or throttled Lambda invocations. Messages are placed on the queue in batches: one flush cycle can close several (sensor type, pond) windows at once, and rather than issuing one send per window, every payload from a cycle is chunked at the ten-entry limit the batch send API imposes and sent in at most that many batch calls, cutting the number of SQS API calls a busy flush cycle makes to roughly a tenth of what one call per window would cost. If that batched send to the queue fails for any reason, the fog gateway's scheduled flush task catches the exception, logs a single line naming the failure, and simply moves on; because the closed windows have already been pulled out of the live buffer before the send is attempted, nothing requeues or retries them, so a single failed publish call means that cycle's readings are dropped rather than merged into a later window, an accepted trade-off given how little a single ten-second window's worth of aggregates costs to lose against the complexity of a local retry or spill-to-disk buffer. The ingestion Lambda that drains this queue processes every record in a batch concurrently rather than sequentially: each record's DynamoDB write is submitted as its own task to a small fixed-size executor, every task is joined, and the individual outcomes are folded into one immutable tally through a stream reduction, so one record failing does not abort the batch early and every record is still attempted regardless of any other record's outcome; the Lambda only re-throws, triggering SQS's own retry of the whole batch, after every write has completed.

DynamoDB backs the store because the dashboard's dominant query, the most recent windows for a given sensor type, maps directly onto a single partition query rather than a table scan once sensor type is chosen as the partition key, a routing property both Bigtable's original design and DynamoDB's own successor architecture treat as the central reason a wide-column store outperforms a general-purpose relational one for this exact read shape [11] [12]. The table's sort key concatenates each window's end timestamp with the reporting pond's identifier, because two ponds can both close a window inside the same ten-second cycle, and a bare timestamp key would let one pond's write silently overwrite the other's. Both Lambda functions run only on demand: the ingestion function fires once per SQS batch its event source mapping delivers, and the dashboard function fires once per incoming API Gateway request, so neither is billed for idle time between windows, which holds here by construction, since a window only produces a queue message when a sensor group actually reported something during it.

C. Dashboard API and Frontend

Locally, the dashboard runs as its own JDK HTTP server serving both the REST API and the static frontend behind the

same module-local request-time dispatcher the fog gateway uses: one registration, one ordered list of path predicates walked per request, and one boundary around every dispatch translating an uncaught exception into a structured 500 response rather than a raw socket close. The two dispatcher classes live in separate packages and are not shared code; they are simply the same design applied independently in both modules, and the dashboard's copy adds a prefix-matching variant used only for its static-asset route. Behind the real API Gateway REST API, the same dashboard logic answers through a distinct Lambda class instead, resolving each request through a two-level lookup (an outer map keyed on HTTP method, an inner map per method keyed on the exact request path) built once in the constructor rather than re-evaluated per request. That Lambda's five routes call directly into the same repository, pipeline-check, and thresholds-proxy logic the JDK-server profile already used, so the query logic itself is not duplicated between the two deployment shapes. A package-private constructor that accepts pre-built AWS clients makes the Lambda directly testable against hand-written fakes of the DynamoDB, SQS, and Lambda client interfaces without needing a running LocalStack instance, and every response it returns, including its 404 and 502 error paths, carries an unconditional wildcard cross-origin header rather than only attaching one to successful responses.

The static frontend renders each pond as an ordinary card listing its five metrics as rows, each drawn against a native HTML meter element rather than a custom-drawn gauge, with a short trend chart underneath for each metric across both ponds, an alert banner summarising every currently active threshold breach across both ponds, and a health strip and record count reflecting the pipeline's own status. A fixed browser-side timer re-fetches the pond snapshot, health status, backend statistics, and every metric's trend history every two and a half seconds, so the cards, alert banner, and trend charts all stay current within a few seconds without any manual refresh. The frontend learns its API base URL from a hidden input element whose value attribute is substituted at upload time with the real API Gateway URL and read once from the page's own script, rather than a build-time environment variable baked into a bundle, a mechanism chosen so the same static files can be uploaded against any deployment's API endpoint without a rebuild step.

D. Security Posture

The EC2 instance hosting the fog gateway and the ten sensor containers exposes only inbound TCP port 8000 through its security group, with no SSH access configured; instance administration goes through Systems Manager rather than a key pair. All three AWS-facing constructors carried over from the development profile (the fog gateway's queue publisher, the ingestion Lambda's client builder, and the local development server's client builder) originally hardcoded a static test credentials provider unconditionally, regardless of which endpoint they were pointed at; the pre-deployment audit described in Section III-D corrected each to check for a LocalStack endpoint override first, and only build the static provider when that override is present. When it is absent, as it always is on the real EC2 instance and inside both real Lambda functions, the AWS SDK's own default credential provider chain resolves automatically to the EC2 instance profile or the Lambda execution role; the dashboard Lambda's client builder, written fresh for the deployment, followed the corrected pattern from the start. That distinction matters specifically because AWS Lambda's managed runtime always injects its own access-key environment variable for the execution role's temporary credentials, so a client builder gated on that variable's mere presence rather than on an explicit LocalStack endpoint would misfire in exactly the environment it is meant to run in; gating on the endpoint instead keeps the static-credentials branch scoped to local development only. Because the dashboard's frontend is served from an S3 bucket while its API answers from a separate API Gateway domain, every browser request between them is cross-origin by the same-origin policy's own definition, which is why the dashboard Lambda's wildcard cross-origin header (Section II-C) is attached unconditionally rather than only to successful responses, so a client-side failure path is never silently blocked by the browser without a visible network error.

III. Implementation

A. Software Stack

The entire system, sensors through both Lambda functions, is Java 17. Every network-facing component (sensor dispatch, the fog gateway, the JDK-server dashboard profile, and the fog gateway's own polling of the AWS SDK) is built on the JDK's standard library alone: its built-in HTTP server for both servers, its built-in HTTP client for outbound calls, and no third-party web framework anywhere in the request path. The AWS SDK for Java v2 supplies the SQS, DynamoDB, and Lambda clients; Jackson supplies JSON parsing throughout, with the fog gateway specifically using Jackson's low-level streaming generator to write every outbound JSON field directly to the response stream rather than building an intermediate object graph first. The dashboard's trend charts use Chart.js, vendored into the static asset directory rather than loaded from a content delivery network, so the page renders identically regardless of whether the browser can reach a third-party host.

B. Automated Test Suite

All 156 tests across the twenty-one files summarised in Table I pass, run with JUnit 5 against hand-written fakes implementing the real AWS SDK client interfaces, with no Mockito and no test anywhere calling a real AWS endpoint or a running LocalStack container. Two files are worth singling out for what they exercise beyond ordinary unit assertions. The sensor suite repeats its core boundary check fifty times rather than once, specifically because a bounded random walk's correctness claim, that it never leaves its configured range, is a property that should hold across many random trajectories, not just the one a single run happens to produce; that suite

TABLE I
AUTOMATED TEST SUITE BY MODULE

Module	Files	Tests	Coverage focus
sensors	2	57	bounded random-walk drift, sensor payload shape, profile registration
fog	10	53	buffering, windowing, alert rules, HTTP ingest over a real socket, batched SQS publishing
backend/processor	3	17	SQS message-to-item mapping, concurrent batch writes, tally correctness
backend/dashboard	6	29	DynamoDB query/pagination, health checks, Lambda dispatch
Total	**21**	**156**	

alone accounts for fifty-three of the fifty-seven sensor-module tests. The buffering suite includes a concurrent stress test that hammers the fog gateway's own merge-based buffering from sixty-four threads simultaneously and asserts afterward that no reading was lost to a race, which is the closest the suite comes to proving the lock-free buffering design in Section II-A under genuine contention rather than only in a single-threaded unit test. A third suite binds the exact HTTP server class the production fog gateway uses to an ephemeral local port and drives it with real HTTP requests over an actual socket for most of its cases, rather than calling the ingest handler directly in-process, the only way to prove that a request body which is not valid JSON at all, as opposed to a well-formed document missing a required field, is rejected with a real 400 status code by whichever layer parses the body first.

C. Continuous Integration

A GitHub Actions workflow runs on every push and pull request, split into two dependent jobs. The first installs a Java 17 toolchain and runs the Maven test goal in each of the four modules in turn. The second job only starts once the first has succeeded; it installs Python 3.12, brings up the full Docker Compose stack (fog gateway, ten sensor containers, and a LocalStack-backed AWS emulation) and runs a separate end-to-end verification script against the live containers, confirming a reading actually reaches DynamoDB through the whole pipeline rather than only checking that individual functions return correct values in isolation. Container logs are captured automatically if the integration job fails, and the stack is torn down unconditionally afterward regardless of outcome, so no containers linger between workflow runs.

D. AWS Deployment and Defects Corrected

The system is provisioned by a reusable Terraform module, applied here as a single terraform apply creating twenty-four resources with no manual AWS CLI step in between. The module's local state file, left over from a previous unrelated deployment through the same module, already tracked a different set of live resources under its default workspace, so a dedicated workspace was created before this project's apply specifically to keep its own state isolated, rather than risk one apply's plan touching resources it does not own. The live account carries a DynamoDB table, an SQS queue, the two Lambda functions described in Section II-B behind an API Gateway REST API, the EC2 instance and Elastic IP hosting the fog gateway and sensor containers, and two S3 buckets, one serving the static frontend and one used only to stage the deployment package.

A pre-deployment audit examined the codebase for three defect classes before any of it reached the live account. The first was a DynamoDB item-count routine that issued a single count-only scan and returned that scan's count directly: a single scan response only covers roughly one megabyte of table data, so once a table grows past that size such a call silently undercounts rather than failing loudly. The fix threads DynamoDB's own pagination cursor across repeated scans in a do-while loop, accumulating each page's count and only stopping once a response reports no further page, so the reported total stays correct regardless of table size. The second was the fog gateway's SQS publishing, which originally issued one send per closed window; because a single ten-second flush cycle can close several sensor-and-pond windows simultaneously, that pattern issues one SQS call per window even when several windows close in the same cycle. The fix batches an entire cycle's payloads into batched sends chunked at that API's ten-entry limit, cutting the number of publish calls a busy cycle makes roughly tenfold. The third, described in Section II-D, was in credential handling: all three AWS-facing client builders carried over from the development profile constructed static emulator-only credentials unconditionally, which on real infrastructure would have overridden the execution-role and instance-profile credentials the deployment depends on and failed every AWS call; each was corrected to build that static provider only when a LocalStack endpoint is configured. None of the three was found live in production; all were caught and corrected during the pre-deployment code audit, before the account described above ever ran the affected code.

E. Live Deployment Verification

Beyond the passing test suite, the deployed system was checked directly against its live endpoints rather than only against local or LocalStack-backed runs. Within roughly a minute of the EC2 instance completing its Docker bootstrap, the health endpoint reported all four of its fields (gateway reachability, queue reachability, ingestion Lambda deployment status, and overall pipeline freshness) true, with the freshest window's age under two seconds, meaning a reading generated on a sensor container was visible through the full pipeline, queue and Lambda included, inside that window. The backend-stats endpoint at the same early check already showed real items landed in the DynamoDB table, and the per-pond endpoint returned genuine per-metric data for both ponds rather than an empty shape. All four of the frontend's static assets returned HTTP 200 directly from the S3 bucket, the API's wildcard cross-origin header was present on live responses, and the hidden input's value had been correctly substituted with the real API Gateway URL at upload time rather than left

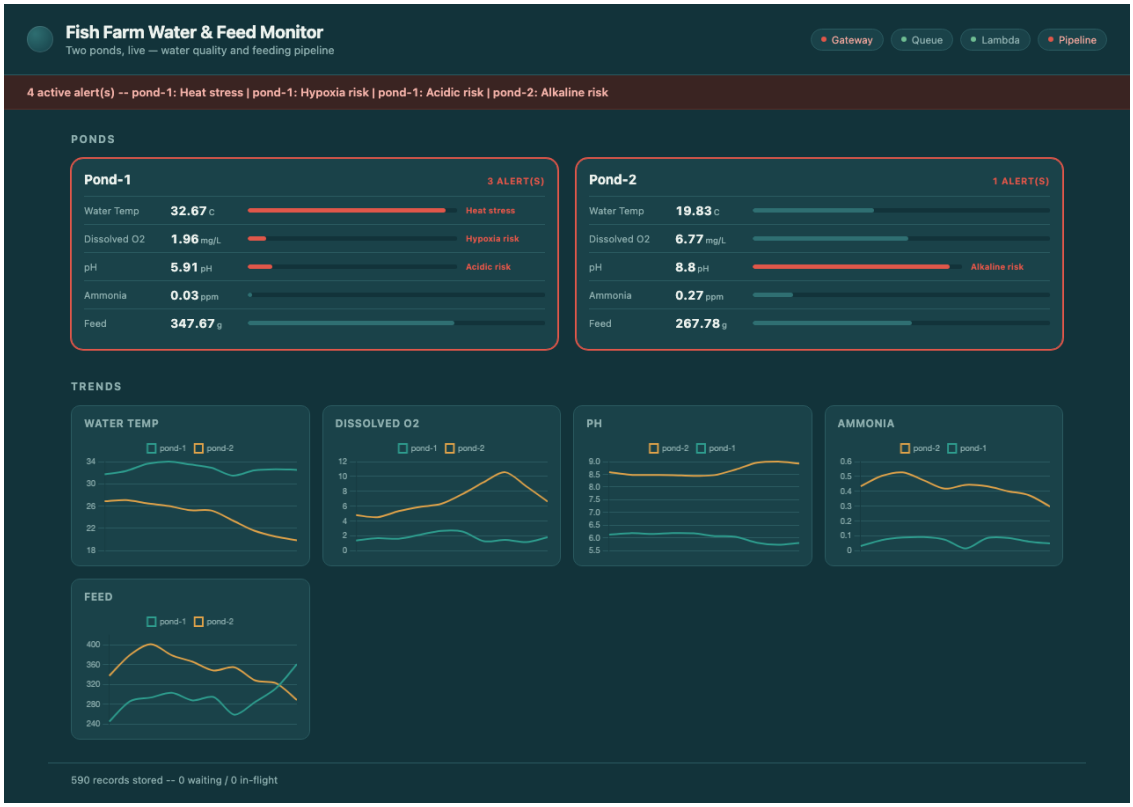

Fig. 2. The deployed dashboard, captured live against the real API Gateway endpoint. Both ponds render real climbing metric values against their configured meter ranges; pond-1 shows three simultaneous alert badges (heat stress, hypoxia risk, acidic risk) and pond-2 one (alkaline risk), matching the threshold rules in Section II-A; the five trend panels beneath plot live per-pond history for each metric, and the footer reports the DynamoDB item count and queue depth returned by the live backend-stats endpoint.

as its placeholder token, confirming the mechanism described in Section II-C actually took effect on the deployed files, not only in the unsubstituted source file.

Fig. 2 captures the system at a later point in its continued operation, after several hundred additional windows had accumulated: the DynamoDB item count has grown into the hundreds, four alert conditions are firing simultaneously and correctly across the two ponds against the exact threshold values in Section II-A, and every trend panel shows a genuine, non-flat history rather than a placeholder or a single data point. Together, the early post-deployment check and this later capture demonstrate not just that the pipeline came up correctly once, but that it continued producing and serving fresh data over a sustained period without manual intervention.

IV. Conclusion

The design choices in Section II were not the only ones available, and two are worth weighing against alternatives that were considered and set aside. Buffering readings in a concurrent hash map with per-key merge, rather than an explicit lock around a shared map or a dedicated single-threaded actor draining a mailbox queue, trades a small amount of conceptual overhead (correctness here depends on understanding exactly what guarantees the merge operation provides and on the accumulator type staying genuinely immutable) for the removal of any single point of lock contention between the many concurrent ingest calls a busy pond generates and the periodic flush. An actor-style design would have made that reasoning easier to state but would have introduced a single draining thread as a throughput ceiling the lock-free design does not have. Choosing DynamoDB with sensor type as the partition key optimises specifically for the dashboard's own dominant read pattern (recent windows for one sensor type) at the cost of making a cross-pond, cross-sensor-type query (everything reported in the last minute, across every metric) comparatively expensive, since no single partition query answers it directly; a relational store with a composite index would have made that particular query cheaper at the cost of the partition-routing benefit DynamoDB provides for the query that is actually exercised on every dashboard load.

The system also has real limitations worth stating rather than glossing over. The alert rules in Section II-A evaluate each metric's window average or, for the pH rules, a single field in isolation; none of the five rules considers a combination of metrics together, so a pond that is simultaneously borderline-low on oxygen and borderline-high on ammonia (a combination materially more dangerous to fish than either condition alone) shows two independent low-severity badges rather than one elevated compound warning. The ten-second window is also fixed at deploy time through an environment variable

rather than tunable per sensor type, even though a fast-moving reading like dissolved oxygen and a slow-moving one like feed dispensed arguably warrant different windowing granularity. Feed dispensed itself is tracked purely as a quantity dispensed per window with no rule attached to it at all, leaving over- or under-feeding relative to stocking density as a condition the alerting layer cannot currently surface, even though it is one of the five sensor types collected.

Future work follows fairly directly from those gaps: a compound-condition rule type that can reference more than one metric at once would close the oxygen-and-ammonia blind spot described above; per-sensor-type window durations would let the fastest-changing metrics be checked more often without forcing every metric onto the same cadence; and a feeding-efficiency rule relating dispensed quantity to a configurable stocking estimate would put the fifth sensor type's data to use in the alerting layer rather than only in the trend charts.

Building the fog gateway's buffering around the concurrent map's own merge operation, rather than an explicit lock, took longer to trust than it took to write. The instinct on every early failure was to reach for a synchronized block, and the sixty-four-thread stress test in the buffering suite did fail once during development, not because the merge operation itself was wrong, but because the accumulator type it was combining still carried one mutable field left over from an earlier draft, so two concurrent merges could each read that field before either had written it back. Tracking the failure down meant reading the JDK's own documentation for that merge method closely enough to notice it only guarantees atomicity when the remapping function is free of side effects, a narrower promise than "thread-safe" sounds like at a glance. Learning to tell that kind of hazard apart from one the compiler would catch, and to write a test adversarial enough to provoke it rather than one that merely exercises the happy path, was the most transferable outcome of the concurrency work, more so than any single line of the eventual fix.

A second, smaller lesson came from the pre-deployment audit described in Section III-D rather than from anything a running test caught. Both the DynamoDB pagination undercount and the unbatched SQS publishing were found by reading the affected code with the specific question of what happens once the underlying table or queue is under real load, not by any test failing; the existing suite passed cleanly both before and after each fix, since every fake table and queue it exercised was small enough that neither defect's symptom ever had room to appear. That gap between a passing suite and a genuinely correct implementation was the clearest single takeaway from building this project: a green test run answers the question the tests happened to ask, not the question of whether the code is actually right at a scale nobody wrote a test for.

The fog tier built here needed no framework beyond the JDK's own HTTP server and no synchronisation beyond a single concurrent map's documented guarantees to turn a ten-sensor stream into a handful of batched cloud writes per window, and Section III's live-endpoint checks confirm that reduction holds up against a real, separately billed AWS deployment rather than only inside a unit test. Every module and test file referenced in this report sits in the project's own source tree; the GitHub repository hosting it is available at [GitHub repository URL].

References

[1] C. E. Boyd, *Water Quality: An Introduction*, 2nd ed. Cham, Switzerland: Springer, 2015.

[2] Food and Agriculture Organization of the United Nations, "The State of World Fisheries and Aquaculture 2022: Towards Blue Transformation," FAO, Rome, Italy, Tech. Rep., 2022.

[3] B. O'Flynn, F. Regan, A. Lawlor, J. Wallace, J. Torres, and C. O'Mathuna, "SmartCoast: A Wireless Sensor Network for Water Quality Monitoring," in *Proc. 32nd IEEE Conf. Local Computer Networks (LCN 2007)*, Dublin, Ireland, 2007, pp. 815–816.

[4] D. S. Simbeye and S. F. Yang, "Water quality monitoring and control for aquaculture based on wireless sensor networks," *Journal of Networks*, vol. 9, no. 4, pp. 840–849, 2014.

[5] C. Encinas, E. Ruiz, J. Cortez, and A. Espinoza, "Design and implementation of a distributed IoT system for the monitoring of water quality in aquaculture," in *Proc. 2017 Wireless Telecommunications Symposium (WTS)*, Chicago, IL, USA, 2017, pp. 1–7.

[6] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12)*, Helsinki, Finland, 2012, pp. 13–16.

[7] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-145, 2011.

[8] D. Lea, "The java.util.concurrent synchronizer framework," *Science of Computer Programming*, vol. 58, no. 3, pp. 293–309, 2005.

[9] Y. K. Ip and S. F. Chew, "Ammonia production, excretion, toxicity, and defense in fish: a review," *Frontiers in Physiology*, vol. 1, p. 134, 2010.

[10] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003.

[11] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, "Bigtable: A distributed storage system for structured data," *ACM Transactions on Computer Systems*, vol. 26, no. 2, pp. 1–26, 2008.

[12] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," *ACM SIGOPS Operating Systems Review*, vol. 41, no. 6, pp. 205–220, 2007.